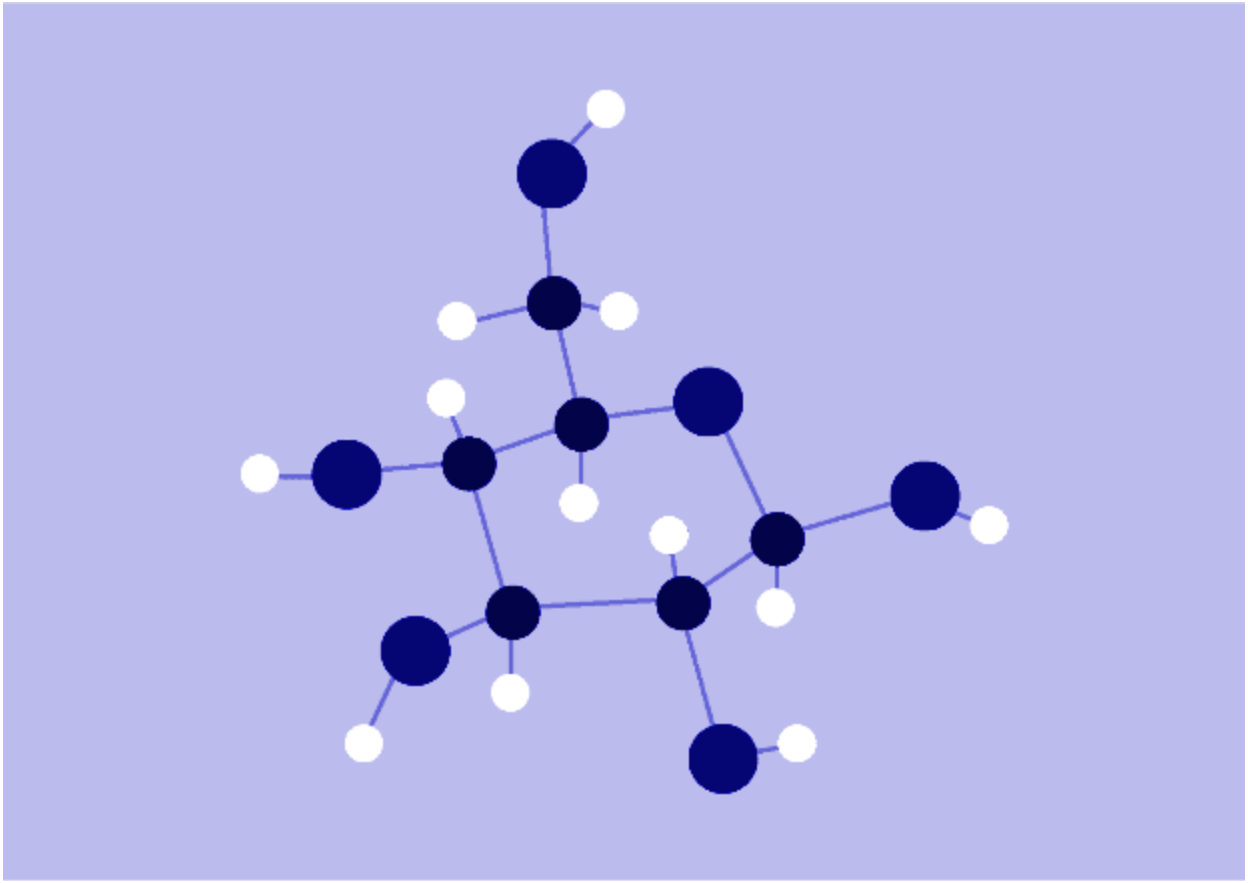
Designbeschreibung

Online Aufgaben-Planungssystem/Selbst-Managementsystem



NI0 - Name Ideas 0

04.09.2026

Autor: Konstantin Fastovski, Felix Weglehner, Elias Kerlin

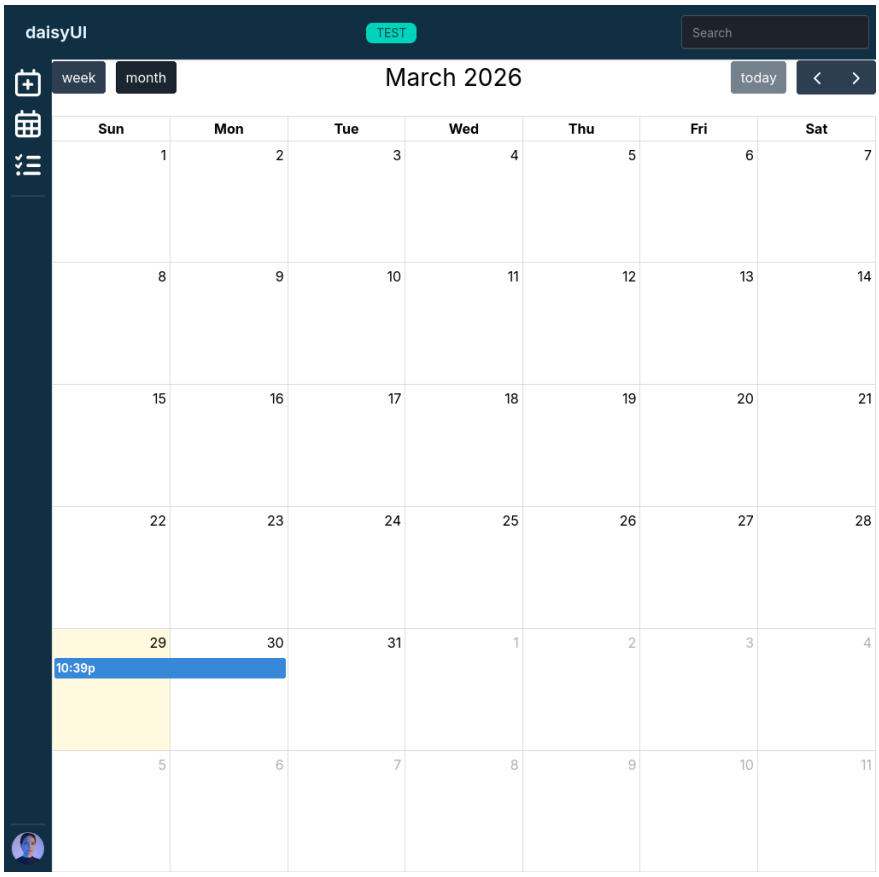
Prüfleser: Paul Berndt

Allgemeines **2**
Produktübersicht **2**
Struktur & Entwurfsentscheidungen **3**
Authentifizierung 4
Server 5
Client 7
Deployment 8
Anhang **9**

Allgemeines

Im Folgenden handelt es sich um eine Beschreibung des bisher entwickelten Prototyps des Aufgaben-Planungssystems/Selbstmanagementsystems. Das Dokument geht sowohl auf die Oberfläche und die Anweisungen zur bisherigen Bedienung als auch auf Modellierungsentscheidungen ein. Diese sind relevant, da auf der Basis dieses Prototyps der Rest der Software aufgebaut werden soll.

Produktübersicht



Die Applikation ist eine Single-Page-Application, auf der der Nutzer die meiste Zeit in einer Aufgabenlisten- oder Kalenderansicht verbringt. Für diese Komponenten ist der Großteil der Fläche auf der Seite vorgesehen. Zwischen den Ansichten kann der Nutzer mithilfe der seitlichen Navigationsleiste navigieren. Die Navigationsleiste oben ist für eine Suchfunktion und ein Banner für den als nächstes auftretenden Termin reserviert.

Das wichtigste bereits funktionierende Feature ist der oberste der Knöpfe auf der seitlichen Navigationsleiste. Dieser öffnet ein Menü zum Erstellen einer neuen Aufgabe, in das der Nutzer Titel, Beschreibung, Start- und Enddatum, Labels und Zeitblöcke einpflegen kann. Die erstellten Aufgaben kann der Nutzer in der Kalenderansicht einsehen. Der Kalender hat mehrere Ansichten, zwischen denen der Nutzer wechseln kann, und verfügt über Navigationsknöpfe zum Blättern durch den Kalender.

Zur Verwaltung der eigenen Identität und zur Konfiguration der Anwendung steht dem Nutzenden ein umfassendes Einstellungsmenü (Settings Modal) zur Verfügung, das als Overlay über der Hauptansicht geöffnet wird. Das Menü gliedert sich in sechs Kategorien, die der Nutzer über eine seitliche Navigation innerhalb des Modals erreicht:

- **Account & Security:** Passwort-Zurücksetzung per E-Mail-Link sowie Aktivierung/Deaktivierung der Zwei-Faktor-Authentifizierung (siehe Anhang).
- **Appearance:** Auswahl des Farbschemas (Light, Dark, System) über visuelle Vorschaukarten (siehe Anhang).
- **Organizations & Profiles:** Übersicht der Organisationsmitgliedschaften mit Rolle und Mitgliederzahl, Auswahl eines primären Kontaktprofils sowie die Möglichkeit, Accounts über eine E-Mail-basierte Merge-Funktion zu verknüpfen (siehe Anhang).
- **Work Preferences:** Konfiguration des individuellen Arbeitsprofils, bestehend aus Arbeitsstunden pro Tag/Woche, maximaler täglicher Arbeitszeit, Pausendauer und -intervall sowie der Option, diese Einstellungen pro Organisation individuell zu überschreiben (siehe Anhang).
- **Notifications:** Verwaltung von E-Mail-Benachrichtigungen (z. B. Task Assigned, Weekly Summary, Organization Updates) und Erinnerungen (z. B. Due Soon, Overdue Tasks, Daily Planning) über individuelle Toggles (siehe Anhang).
- **Data & Privacy:** Export aller Nutzerdaten im JSON-Format sowie die Möglichkeit, den eigenen Account inklusive aller zugehörigen Daten unwiderruflich zu löschen (mit zweistufiger Bestätigung) (siehe Anhang).

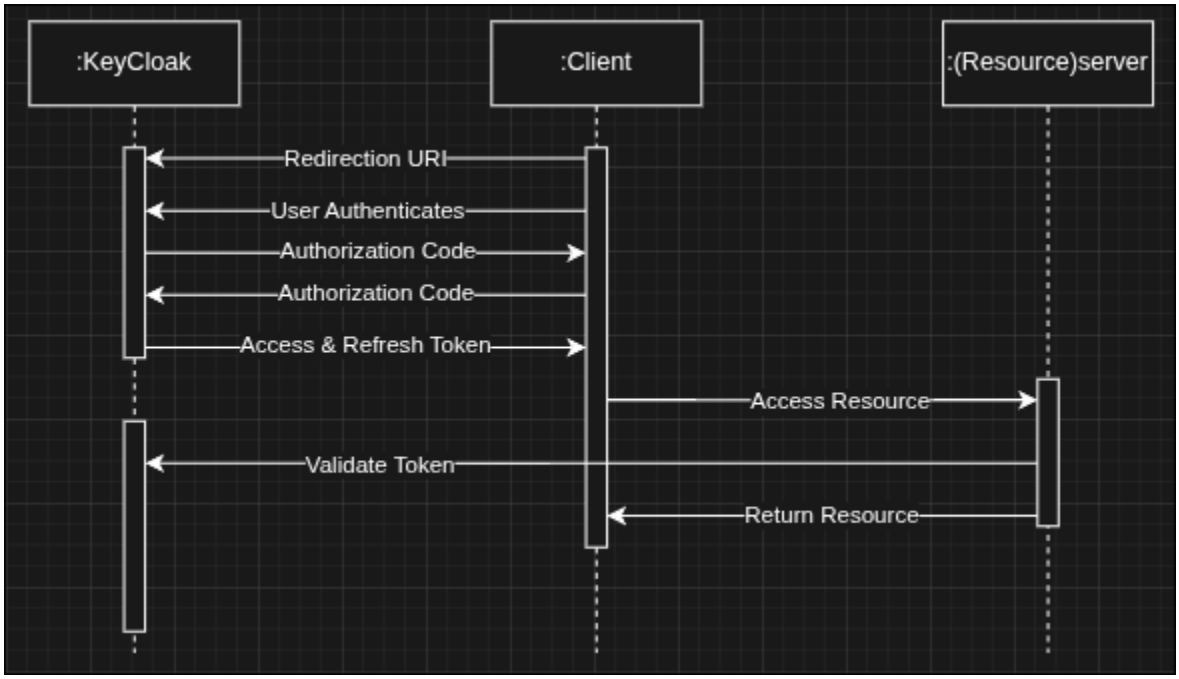
Struktur & Entwurfsentscheidungen

Chronoscope wird als typische SPA (Single-Page-Application) entwickelt.

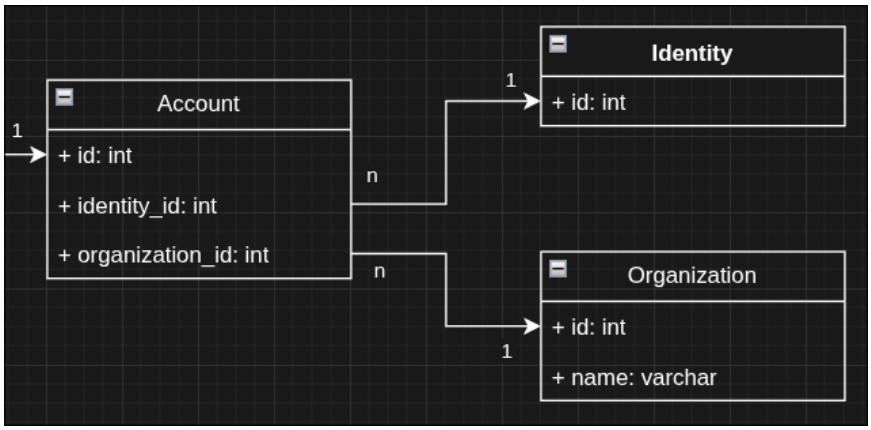
Authentifizierung

Die Authentifizierung des Nutzers erfolgt durch eine Weiterleitung an unseren Identity Provider Keycloak mithilfe des standardisierten Authentifizierungsverfahrens OIDC und ist im folgenden Diagramm kurz beschrieben. Für zusätzliche Informationen ist hier der RFC 6749 zu erwähnen, der das darunterliegende Protokoll OAuth 2 beschreibt.

Der Client nutzt hierfür eine Bibliothek, welche die Tokenverwaltung übernimmt. Der Server nutzt die Spring-Bibliothek für Resource Server zur Verifizierung des Access Tokens.



Folgendes Schaubild veranschaulicht die im Endprodukt angestrebte Datenhaltung bezüglich Account und Identität:



Dabei repräsentiert ein Account einen Account in Keycloak. Ein solcher Account kann einer Organisation zugeordnet sein, oder keinem, wenn ein privater Account genutzt wird. Um über mehrere Accounts und somit auch Organisationen hinweg zu arbeiten, können mehrere Accounts miteinander verknüpft werden. Dabei sind Accounts an eine Identität gebunden, die i. d. R. eine bestimmte Person repräsentiert. Die Identität wird automatisch erstellt und ermöglicht im Hintergrund die Abfrage der einzelnen Organisation einer Identität bzw. Person. So kann die Anmeldung via Single-Sign-On (SSO) z. B. indirekt über den Identity Provider einer Organisation in Keycloak verwaltet werden, aber trotzdem eine angenehme account- und somit organisationsübergreifende Anwendung gewährleistet werden. Der Anwender kann sich dann mit einem beliebigen, mit seiner Identität verknüpften Account anmelden und erhält die Möglichkeit zur Einsicht und Verwaltung von Terminen über alle verknüpften Accounts.

Server

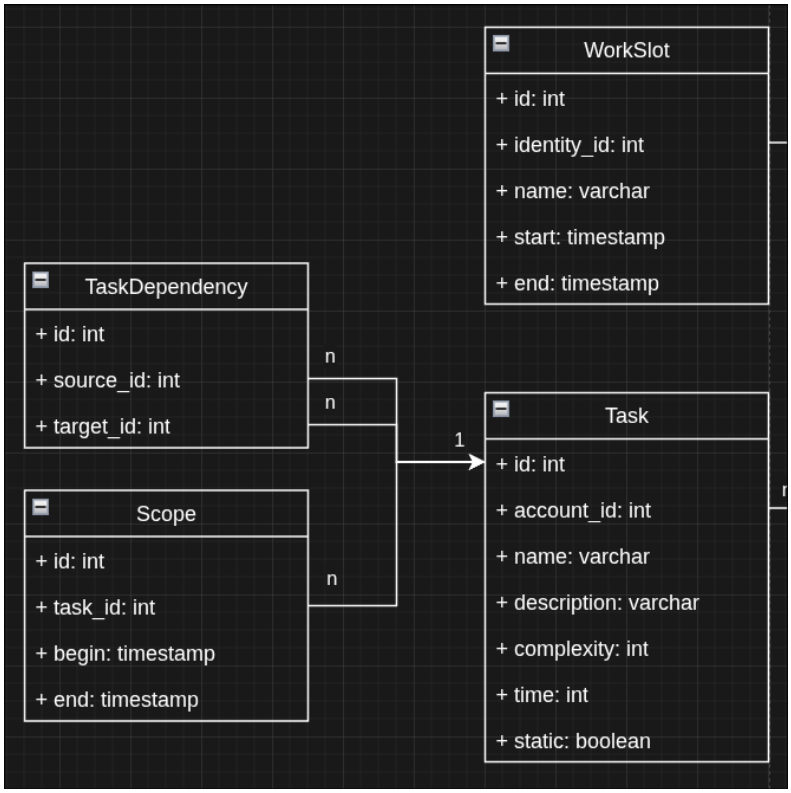
Der Server dient hier lediglich zur Authentifizierung sowie zur Bereitstellung und Modifikation von Daten. Entwickelt als Java-Spring-Anwendung kommuniziert dieser mit der Datenbank, um die Daten zu persistieren. Anfragen werden über eine REST API (Representational State Transfer Application Programming Interface) an den Server geschickt. Daten werden hierbei als JSON formatiert.

Die Spring Anwendung ist aufgeteilt in drei Ebenen: Controller, Service und Repository. Das typische Schichtenmodell isoliert und abstrahiert den Zugriff über externe Schnittstellen (REST), die eigentliche Business-Logik und den Zugriff auf die Datenbank. Die Daten, welche über diese Ebenen transportiert werden, werden durch mehrere

Klassen repräsentiert. Auf der Controller-Seite werden die Informationen durch DTOs (Data Transfer Objects) repräsentiert. Dabei sollte jeder Endpunkt eigene Request- und Response-DTOs besitzen. Die Service- und Repository-Schicht arbeiten mit DAOs (Data Access Objects). Diese repräsentieren die Datenhaltung in der Datenbank. Bei der Kommunikation von Controller und Service Schicht müssen Objekte ggf. umgewandelt werden. Diese Umwandlung bzw. vermeintliche Duplizierung von Daten ermöglicht eine einfache Änderung und Abwandlung von z. B. Informationen, die über einen Endpunkt bereitgestellt werden.

Der Algorithmus zur automatischen Terminplanung läuft on-demand und stateless ab. Der Server muss keine Daten längerfristig im Speicher verwalten bzw. irgendwelche Timer prüfen, z. B. via Redis.

Folgendes Diagramm beschreibt den Aufbau und die Beziehungen der Daten.



Die Aufgabe des zentralen Algorithmus ist hier das Verarbeiten der vom Nutzer definierten Tasks und Erstellen von sogenannten “Scopes”. Dies ist der Begriff für die Arbeitsslots im Projekt Chronoscope. Der Nutzer kann indirekt WorkSlots festlegen. Das sind Zeitslots, in denen er arbeiten möchte. Die Scopes werden dann in den WorkSlots

verteilt. Die WorkSlots haben außerdem einen abstrakten Typ (Name-Attribut im Diagramm). Die Tasks können bestimmten WorkSlot-Typen zugewiesen werden. Standardmäßig können die mit der Identität verknüpften Organisationsnamen als WorkSlot-Typ ausgewählt werden. So können die Arbeitszeiten für verschiedene Organisationen getrennt konfiguriert werden, während trotzdem die Möglichkeit besteht, das System der WorkSlot-Typen durch eigens erstellte Typen zu erweitern.

Client

Die Darstellung der Inhalte oder Bereitstellung von Möglichkeiten zur Modifikation ist Aufgabe des Frontends. Dieses ist als Angular-Anwendung entwickelt und folgt, was die Dateistruktur betrifft, den Best Practices für Angular-Applikationen. Der Eintrittspunkt ist das Modul `src/app/app`. In dessen Template (also `src/app/app.html`) wird die Grundstruktur der Seite festgelegt. Im Falle dieses Prototyps werden die Komponenten `src/app/components/topbar/topbar`, `src/app/components/navbar/navbar` und `src/app/components/calendar-view/calendar-view` platziert. Die Stilisierung erfolgt mithilfe von TailwindCSS, anstelle von traditionellen CSS-Dateien. Diese Entscheidung ermöglicht unter anderem die Nutzung von Plugins wie z. B. DaisyUI, das viele vorgefertigte Komponenten enthält und den Entwicklungsprozess beschleunigt. TailwindCSS ermöglicht außerdem den effizienten Einstieg in das Webdesign für weniger erfahrene Teammitglieder.

Für den Kalender wählen wir den Fullcalendar von Adam Shaw, da er kostenlos und reich an Features ist. Da das Event-Modell, das von Fullcalendar verwendet wird, nicht nativ erlaubt, Aufgaben so zu planen, wie wir es für die Anwendung vorgesehen haben, gibt es die Klasse `src/app/model/task`. Durch den Dialog zum Erstellen einer neuen Aufgabe wird eine solche Task erstellt, und der Dienst `src/services/task.service` übersetzt die Eingabe entsprechend in Event-Objekte, die der Kalender versteht, und stellt sie an die Abonnenten des Observables bereit.

Für die endgültige Anwendung wird hier NgRx angestrebt. NgRx ist eine Angular Bibliothek, die das Redux-Pattern umsetzt und so die Verwaltung komplexerer Daten sowie die Kommunikation zum Server mittels Seiteneffekten vereinfacht. Die Datenhaltung würde hier nicht mehr über reine Services passieren (wie in der Designstudie), sondern über sogenannte Stores.

Einstellungsmenü (Settings Modal)

Das Einstellungsmenü ist als eigenständige Komponente `src/app/components/settings-modal/settings-modal` implementiert. Es wird als modales Overlay über der Hauptansicht angezeigt und von der Navbar aus geöffnet.

Das Modal selbst verwaltet die aktive Kategorie über ein Signal (`activeCategory`). Die sechs Kategorien werden in einem Array definiert und in der seitlichen Navigation per `@for` gerendert. Im Inhaltsbereich wird per `@switch` die jeweils passende Sektions-Komponente eingeblendet.

Jede Kategorie ist als eigene Standalone-Komponente unter `src/app/components/settings-modal/sections/` realisiert. Dieses modulare Design ermöglicht die unabhängige Entwicklung, das Testen und das Erweitern einzelner Sektionen, ohne die übergeordnete Modal-Logik anzupassen.

Für das visuelle Layout wird ein gemeinsamer Satz von CSS-Klassen (`settings-card, settings-card-title, settings-card-desc`) genutzt, die zentral in `src/styles.css` als `@layer components` definiert sind. Dies gewährleistet ein einheitliches Erscheinungsbild über alle Sektionen hinweg.

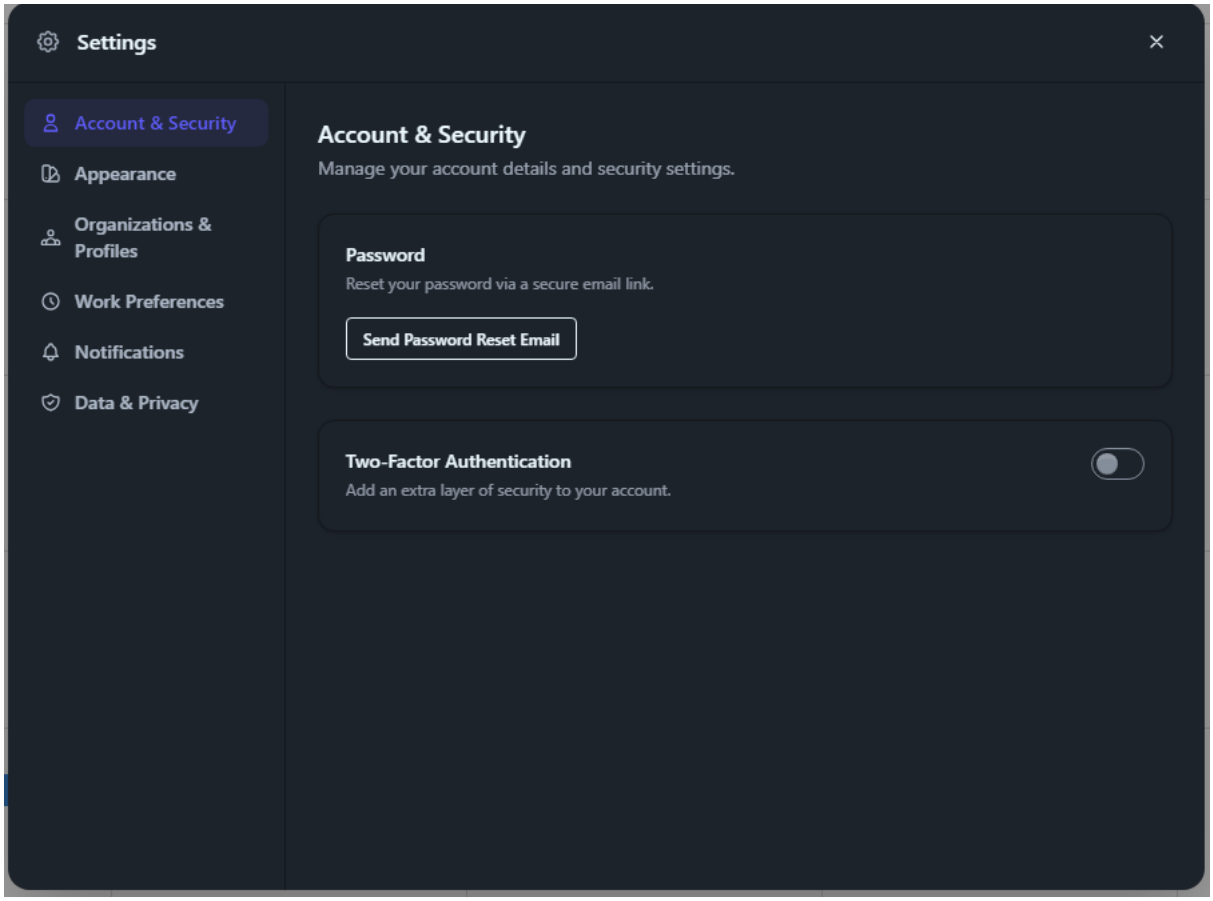
Deployment

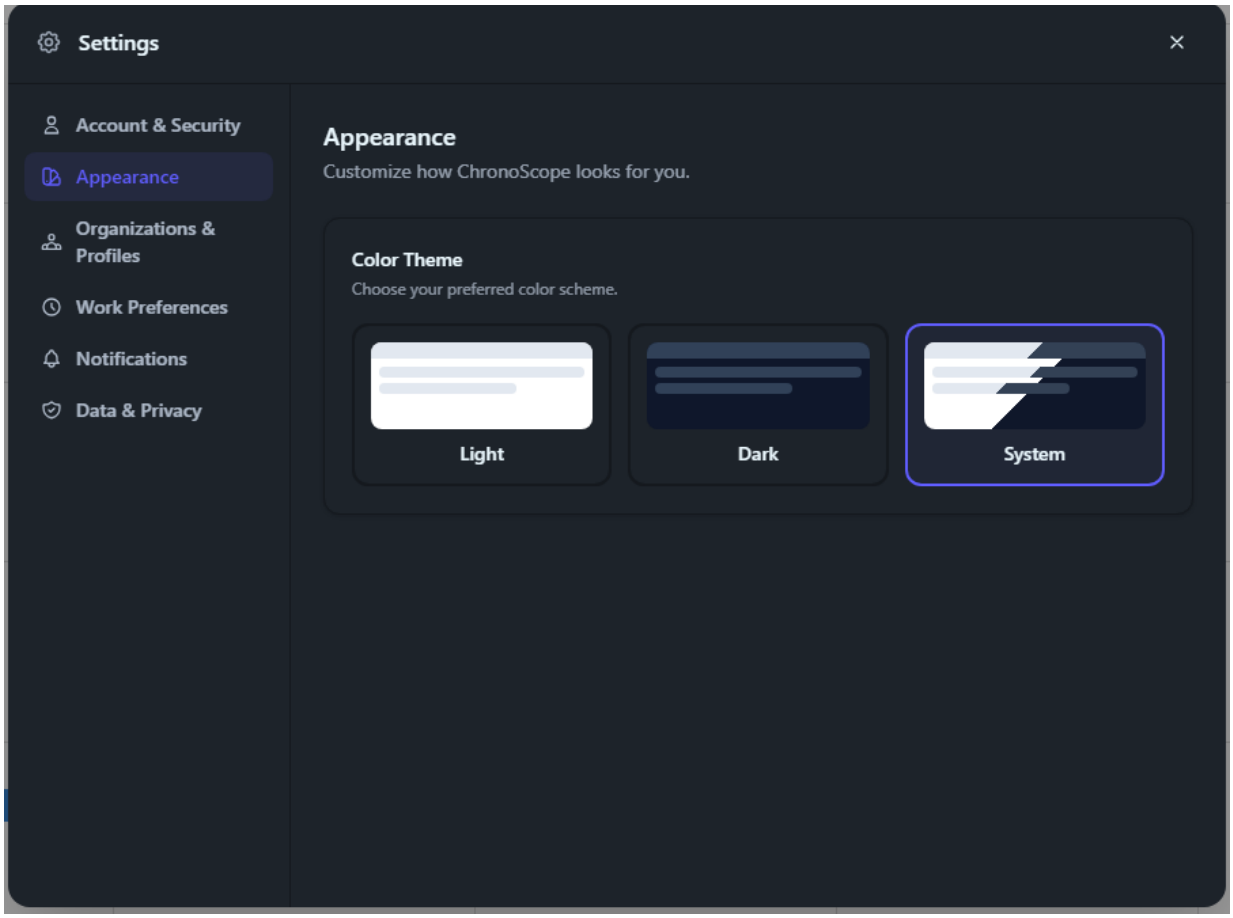
Die Backend-Anwendung wird durch eine GitHub-Pipeline als Docker-Container gebaut und dem Server bereitgestellt. Dieser lässt das Backend innerhalb einer virtuellen Umgebung laufen. Damit nicht die gesamte Spring-Anwendung im Internet steht, werden relevante Endpunkte über einen Nginx-Proxy geleitet.

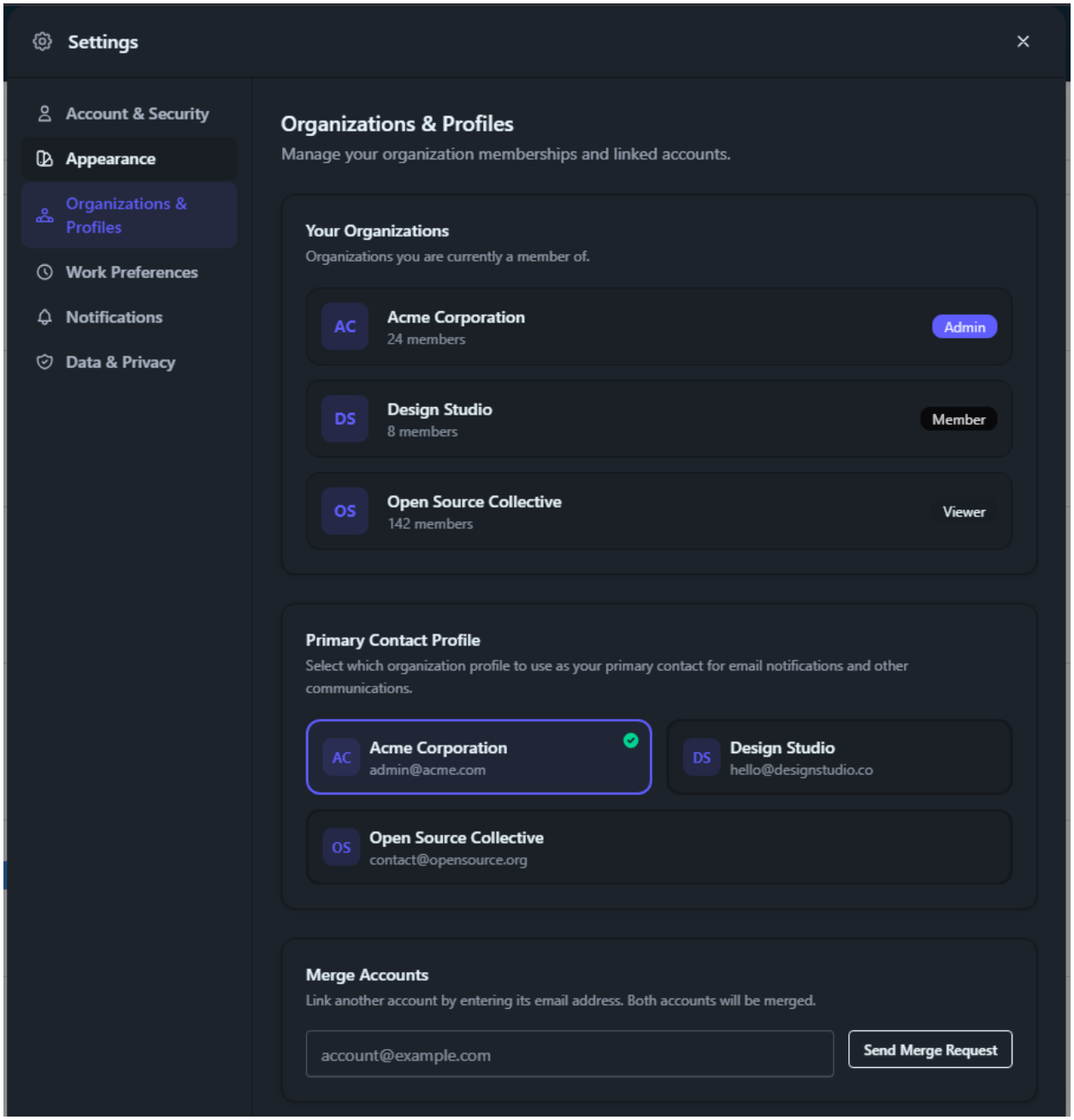
Die Client-Anwendung wird als statische Anwendung (ohne die Notwendigkeit eines laufenden Servers) gebaut und von Nginx ausgeliefert.

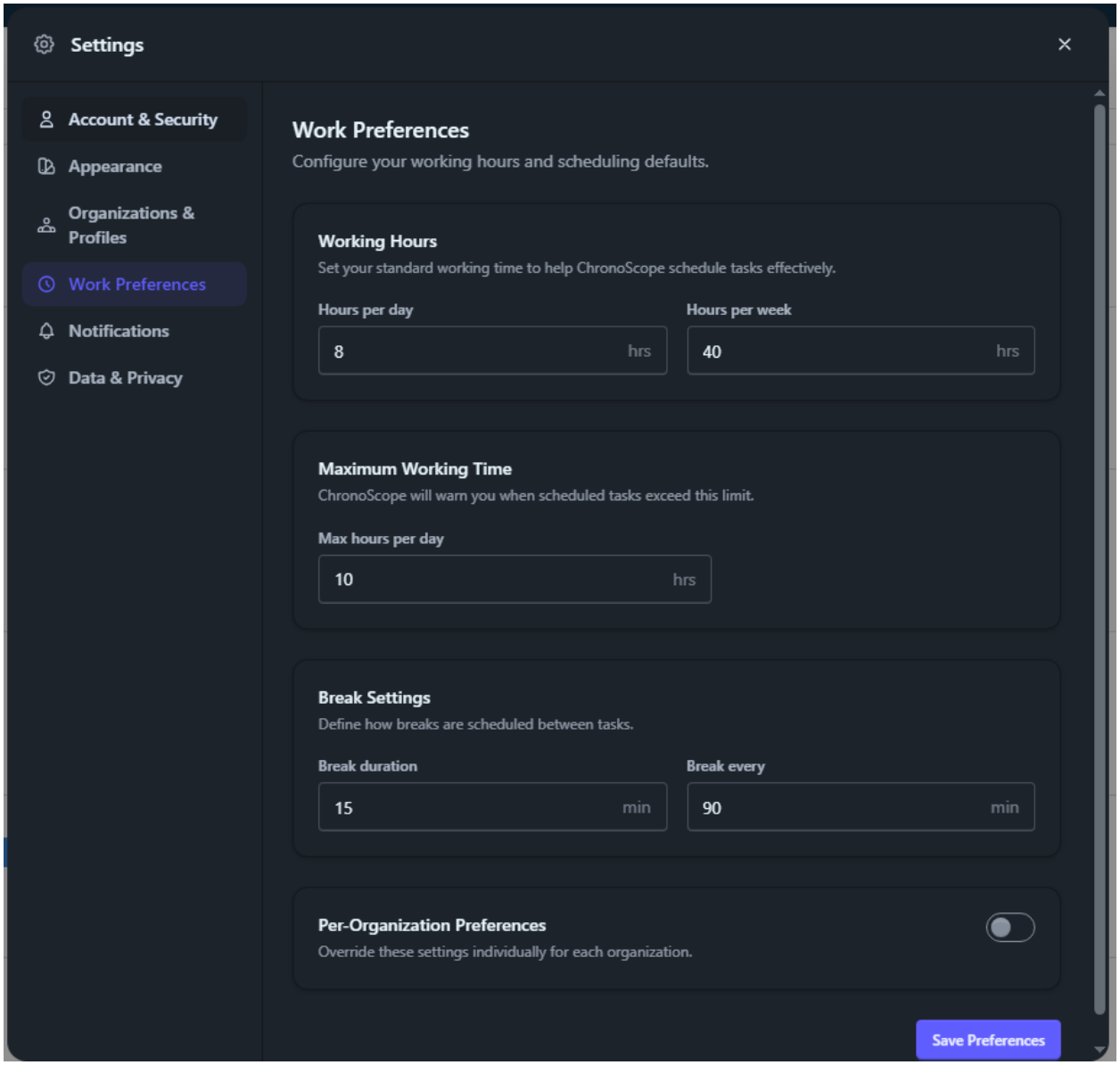
Das Bauen sowohl des Backends als auch des Frontends geschieht automatisch beim Mergen in den Main-Branch.

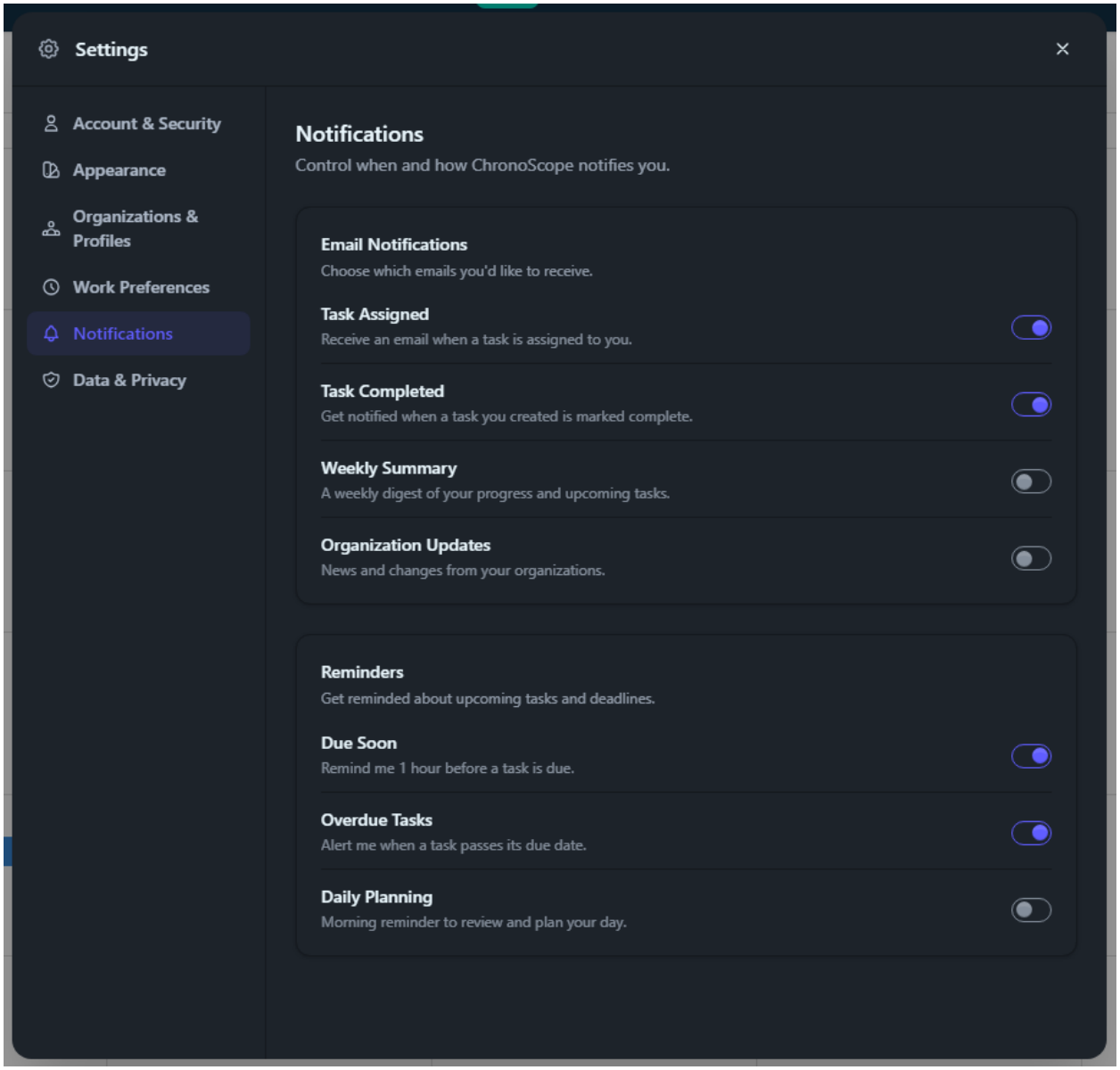
Anhang

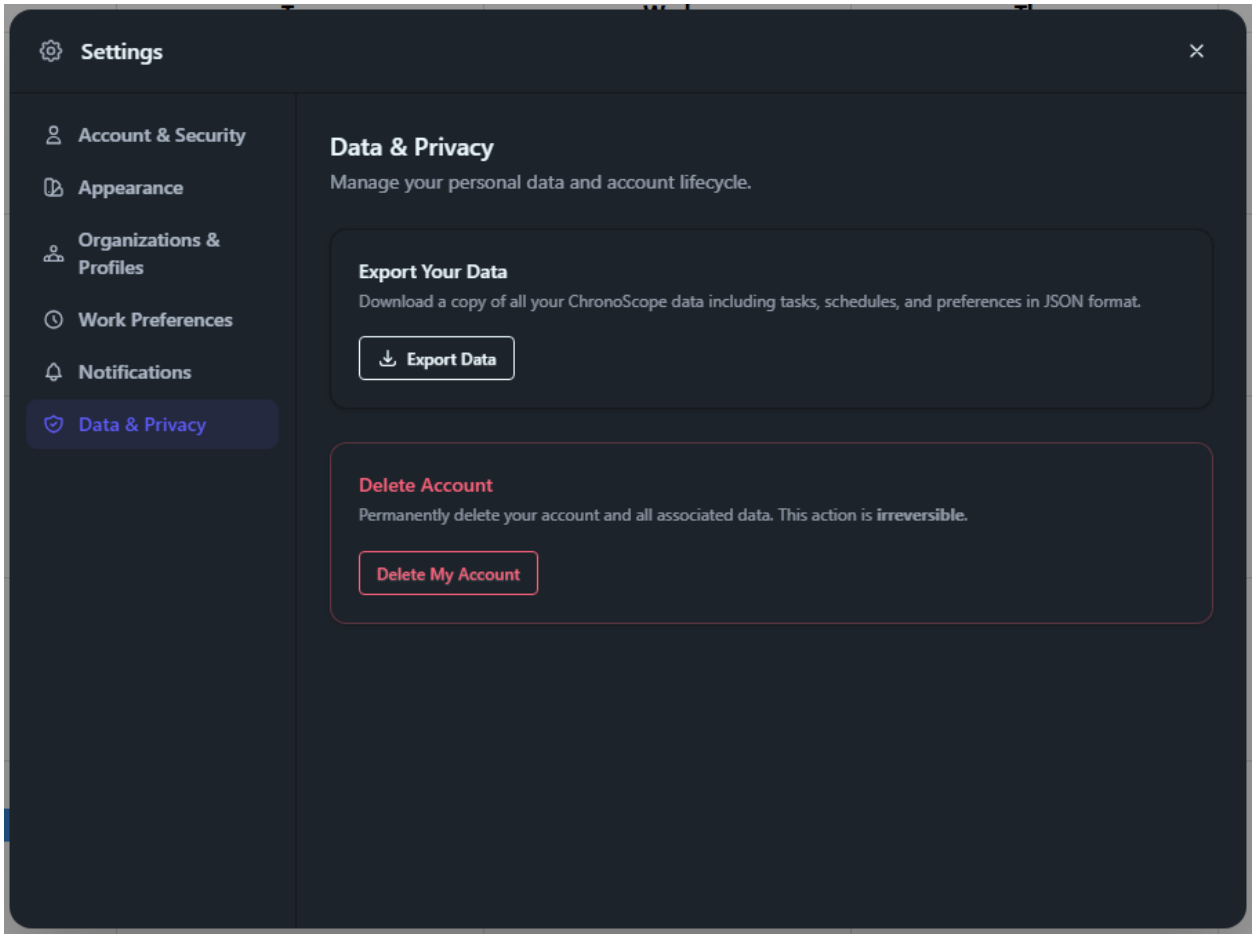

Settings
Account & Security
Appearance
Organizations & Profiles
Work Preferences
Notifications
Data & Privacy
Account & Security
Manage your account details and security settings.
Password
Reset your password via a secure email link.
Send Password Reset Email
Two-Factor Authentication
Add an extra layer of security to your account.


Settings
Account & Security
Appearance
Organizations & Profiles
Work Preferences
Notifications
Data & Privacy
Appearance
Customize how ChronoScope looks for you.
Color Theme
Choose your preferred color scheme.
Light
Dark
System


Settings
Account & Security
Appearance
Organizations & Profiles
Work Preferences
Notifications
Data & Privacy
Organizations & Profiles
Manage your organization memberships and linked accounts.
Your Organizations
Organizations you are currently a member of.
AC
Acme Corporation
24 members
Admin
DS
Design Studio
8 members
Member
OS
Open Source Collective
142 members
Viewer
Primary Contact Profile
Select which organization profile to use as your primary contact for email notifications and other communications.
AC
Acme Corporation
admin@acme.com
DS
Design Studio
hello@designstudio.co
OS
Open Source Collective
contact@opensource.org
Merge Accounts
Link another account by entering its email address. Both accounts will be merged.
account@example.com
Send Merge Request


Settings
Account & Security
Appearance
Organizations & Profiles
Work Preferences
Notifications
Data & Privacy
Work Preferences
Configure your working hours and scheduling defaults.
Working Hours
Set your standard working time to help ChronoScope schedule tasks effectively.
Hours per day
8
hrs
Hours per week
40
hrs
Maximum Working Time
ChronoScope will warn you when scheduled tasks exceed this limit.
Max hours per day
10
hrs
Break Settings
Define how breaks are scheduled between tasks.
Break duration
15
min
Break every
90
min
Per-Organization Preferences
Override these settings individually for each organization.
Save Preferences


Settings
Account & Security
Appearance
Organizations & Profiles
Work Preferences
Notifications
Data & Privacy
Notifications
Control when and how ChronoScope notifies you.
Email Notifications
Choose which emails you'd like to receive.
Task Assigned
Receive an email when a task is assigned to you.
Task Completed
Get notified when a task you created is marked complete.
Weekly Summary
A weekly digest of your progress and upcoming tasks.
Organization Updates
News and changes from your organizations.
Reminders
Get reminded about upcoming tasks and deadlines.
Due Soon
Remind me 1 hour before a task is due.
Overdue Tasks
Alert me when a task passes its due date.
Daily Planning
Morning reminder to review and plan your day.


Settings
Account & Security
Appearance
Organizations & Profiles
Work Preferences
Notifications
Data & Privacy
Data & Privacy
Manage your personal data and account lifecycle.
Export Your Data
Download a copy of all your ChronoScope data including tasks, schedules, and preferences in JSON format.
Export Data
Delete Account
Permanently delete your account and all associated data. This action is irreversible.
Delete My Account